

MAE 3780 Mechatronics, Robot Project Final Report

Group 18

Lilly Varon, Lgv22. Garrett Smith, Gls246. Matthew Heering, Mjh449.

1. Robot Design and Strategy Overview

Our robot was built using the stock chassis for the frame and the two modified servos as driving wheels, all provided by the lab. Each motor was driven by an L9110H H-bridge. We designed two 3D-printed arms that rigidly attach to the chassis (as shown in figure 1). These arms maximise the usable cube collecting area. For sensors we used the stock TCS3200 color sensor (provided by the lab) on the bottom of the chassis and two QTI sensors at the front left and right of the 3d printed arms to detect the black border. All of these were connected directly to the Arduino UNO r3.

The basic idea was to sweep back and forth across the field rather stochastically, collecting cubes and disrupting the opponent robot as we went. At the start of each match, we plug in the DC power to the Arduino. The robot immediately takes color readings to initialize a starting color and begins driving forward. If a large color distance is detected, the robot continues driving forward for about 1 second before abruptly turning and moving back towards the center. The turns are offset using a boolean operator. This way, the robot turns left all odd color changes (1st, 3rd, 5th color change) and turns right all even color changes (2nd, 4th, 6th color change). The two QTI sensors were implemented to assure that if only one sensor is triggered, the robot turns in the opposite direction and if both sensors are triggered, the robot turns a full 180°. See appendix D for our flowchart, which describes the idea well.

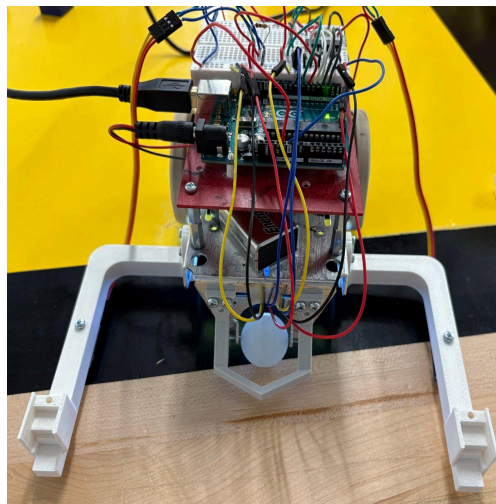


Figure 1. Competition day robot with fixed 3D-printed extensions.

2. Design Process Reflection

We originally planned to use two servo-actuated gates that could swing open to act as an extension of our arms in order to have a greater sweeping area. After the robot swept cubes for a set period of time

we planned to close the gates to trap blocks inside. Near competition day we unfortunately realized we had misinterpreted the size constraints on the extended version of the robot outlined in the rules.

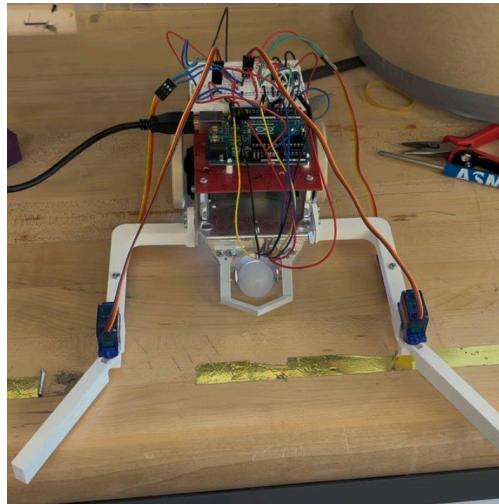


Figure 2. Original design with servo-actuated arms open.

When we put the robot in the size box late in the build period, the open arms extended beyond the 12-inch diameter cylinder limit. Without enough time to redesign the arms without the servo holders, we got rid of the extension arms and kept the fixed extension arms and bolted them to the chassis as they were. The gates shown in appendix A and C were the intended closing mechanism.

The milestones kept us on track for the final competition day. Milestone 2, 3, and 4 were helpful because most of the concepts it was testing were vital to our competition strategy. One thing we spent a lot of time tuning was the sensor thresholds because the color sensor and QTI readings shifted a lot depending on lighting and where we tested.

3. Competition Analysis

We won our round robin group with 5 wins and 1 loss and advanced to single elimination. The sweep worked well against most robots because we were able to grab cubes in the beginning and then push the other robot until they lost their cubes. One thing we didn't anticipate was that the rigidity of the 3d printed extensions let us knock opponents off the board when we made contact, which caused them to lose cubes they had already gathered. That wasn't part of the plan but it helped us advance through the rounds.

In the bracket we won the first match but lost in the second round to a robot that designed their own frame shaped exactly to the 8x8 size constraints. We couldn't sweep around it, and every time we made contact we lost our position and had to reorient. Additionally, their design retained cubes well through a brush sweeping mechanism.

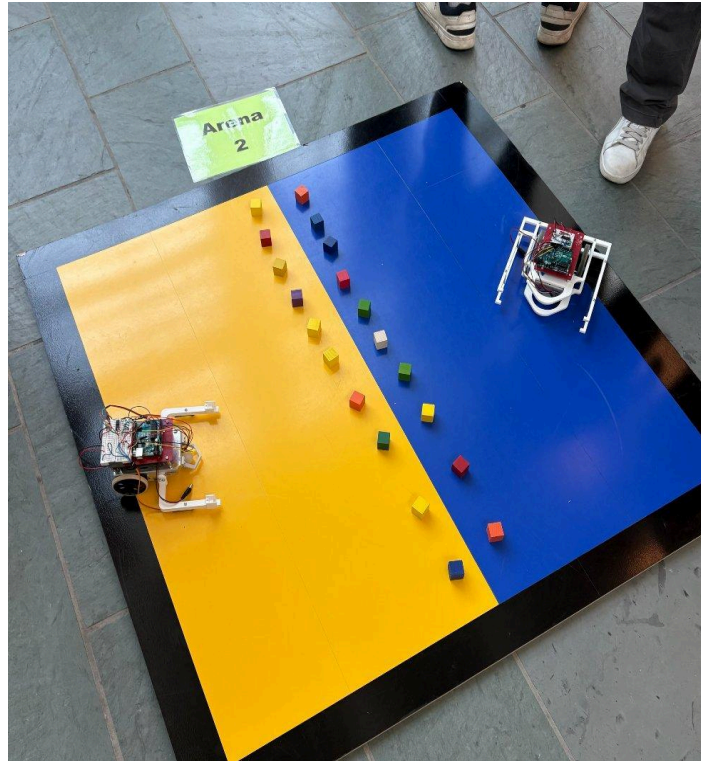


Figure 3. Our robot (yellow side) during a match in the Atrium.

4. Conclusions

We made it out of round robin which was exciting, but losing in the bracket showed we still had two obstacles, what to do against a robot that cannot lose its cubes, and how to stop losing cubes we had already gathered.

If we did this again, we would change our design for the servo arms. This would allow us to be within the size constraint from the beginning, instead of having to make a last minute decision to not use the gates. The added arms could have helped us contain the cubes we lost while turning or bumping into other robots.

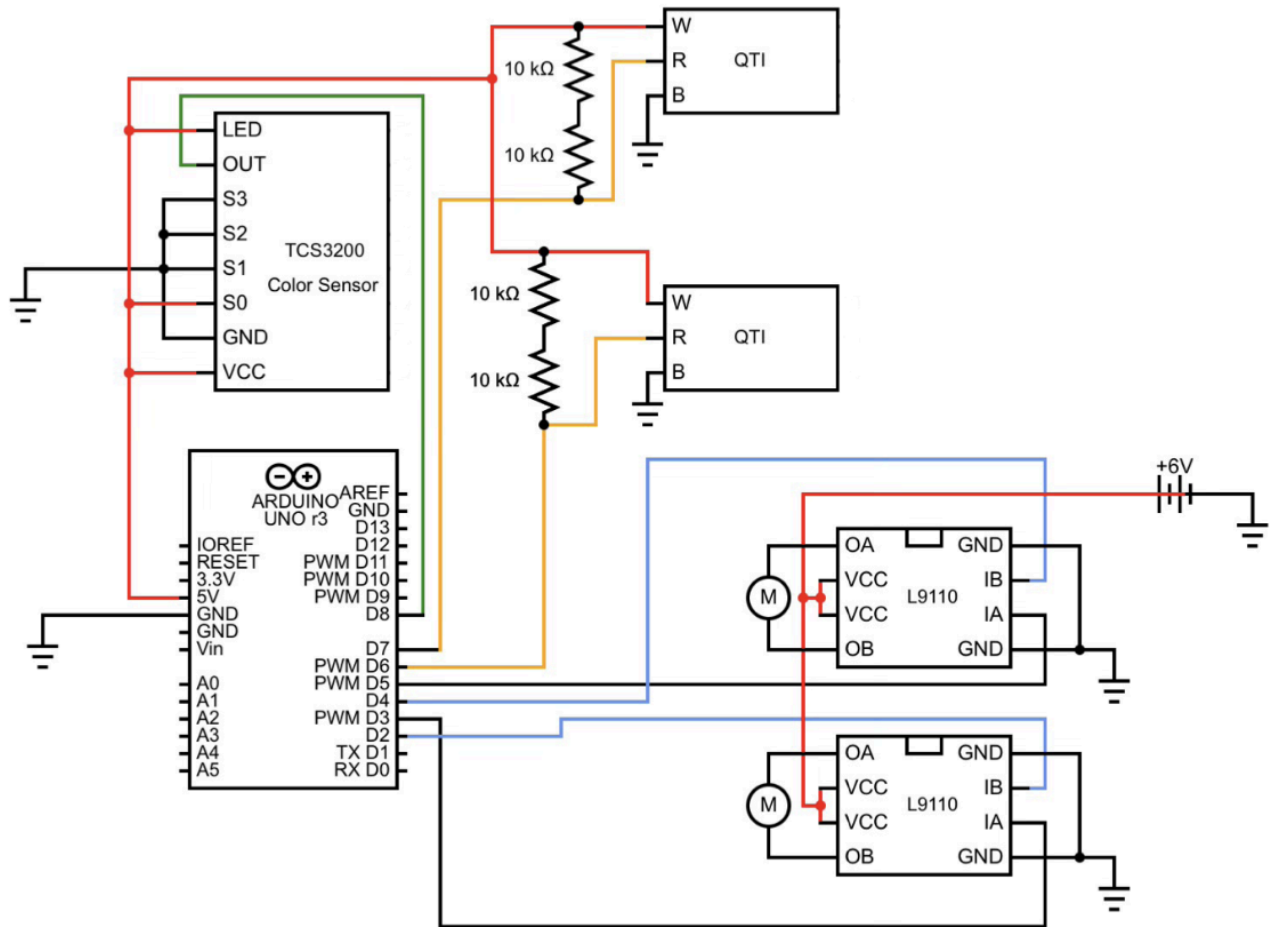
For future groups, two pieces of advice would be to check your dimensions against the size box earlier in the process, and to check your sensors against different light because the lighting in Duffield is noticeably different from the lab.

Appendix A

Bill of Materials						
Name	Part #	Description	Supplier	Quantity	Unit Price	Total Cost
Left Arm	001	ABS 3D-printed left collection arm rigidly attached to robot chassis.	RPL	1	\$8.15	\$8.15
Right Arm	002	ABS 3D-printed right collection arm rigidly attached to robot chassis.	RPL	1	\$8.15	\$8.15
Left Gate	003	ABS 3D-printed swing gate to extend the robot arm reach. Was not used on the final build.	RPL	1	\$4.48	\$4.48
Right Gate	004	ABS 3D-printed swing gate to extend the robot arm reach. Was not used on the final build.	RPL	1	\$4.48	\$4.48
Duct Tape	005	Adhesive for part mounting.	Walmart	1	\$5.77	\$5.77
Front Guard	006	PLA 3D-printed guard for front wheel.	RPL	1	\$2.70	\$2.70
Positional micro servo	007	Additional positional micro servo to operate gates. Was not used on the final build.	Lab	1	\$3.00	\$3.00
Total:				6		\$36.73

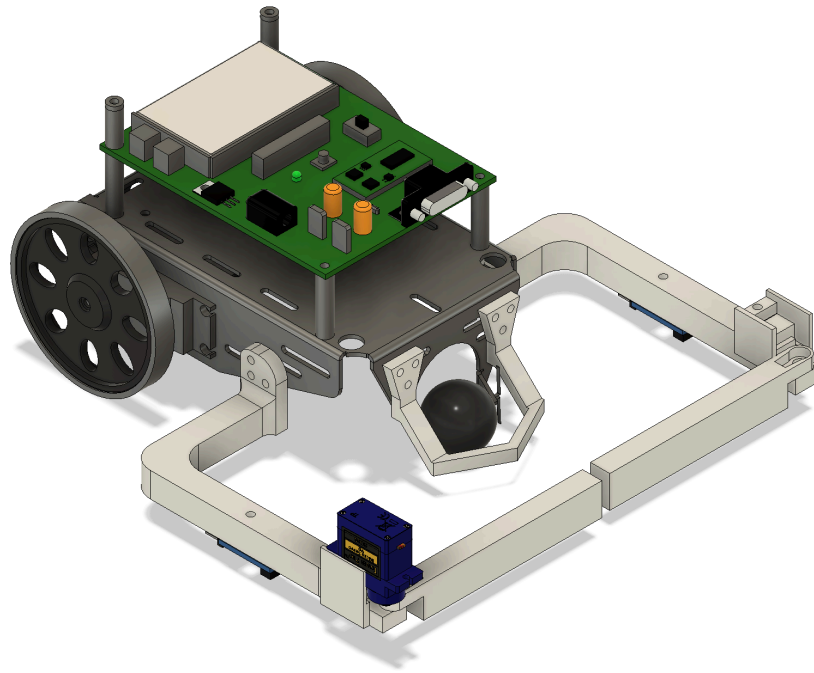
Appendix B

Circuit Diagram

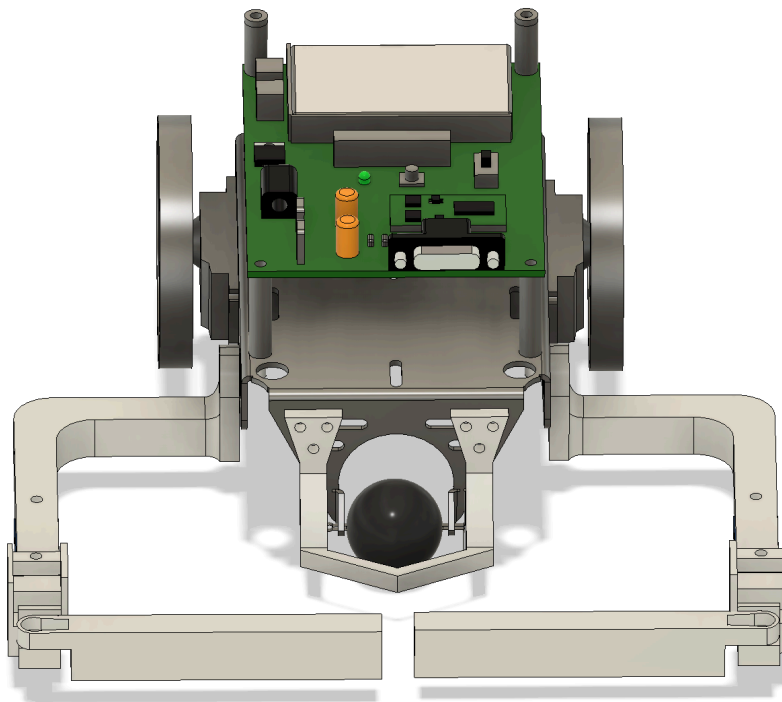


Appendix C

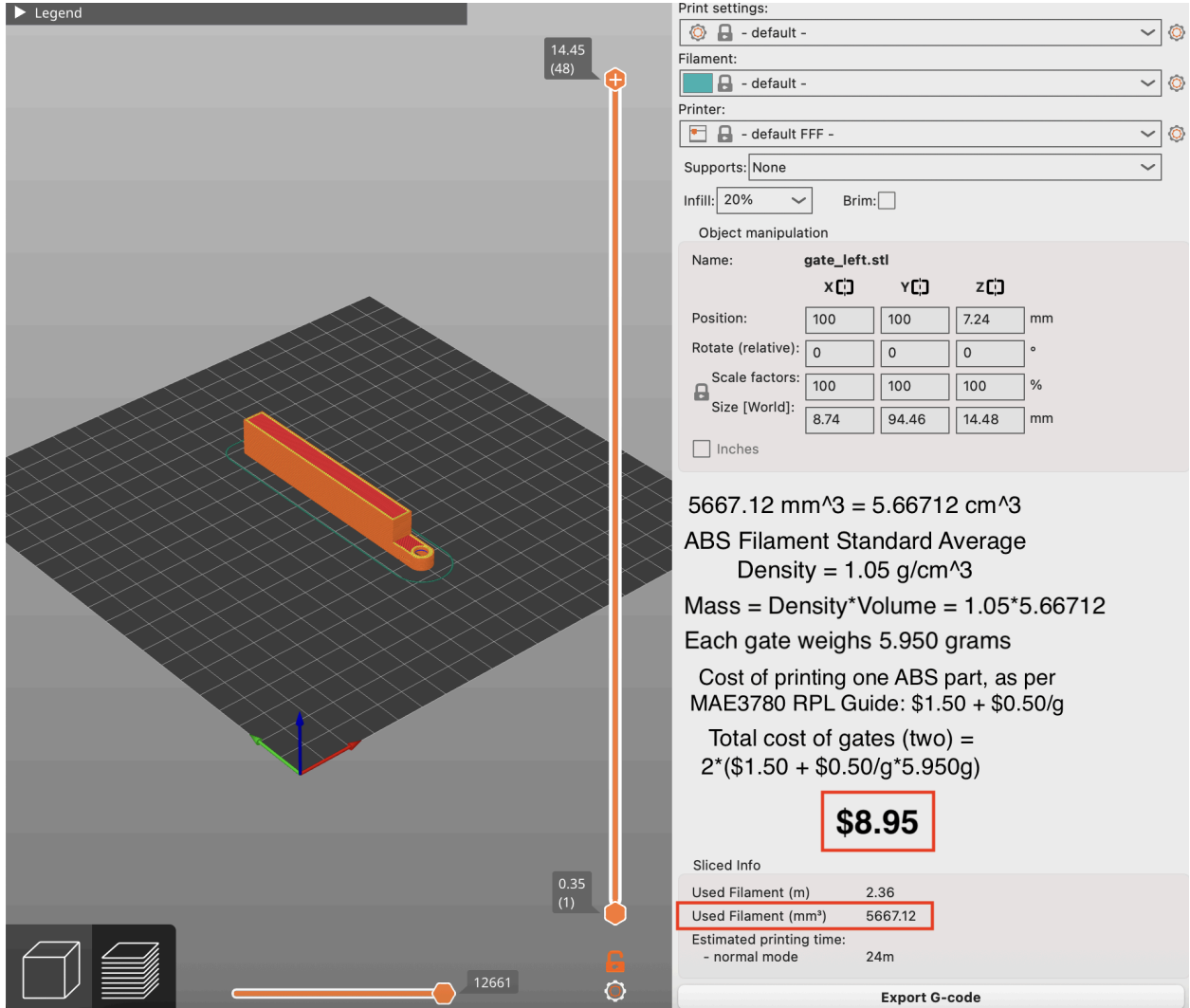
CAD Isometric View



CAD Front View



Weight + Cost of (2) Gates Shown in BOM (Appendix A)



The screenshot displays a 3D printing software interface. On the left, a 3D model of a gate is shown on a grid. A vertical orange line with a crosshair at the top and bottom indicates a measurement of 14.45 (48) mm. The bottom left shows a legend, a stack of layers, and a progress bar at 12661. The right panel contains settings and calculations.

Print settings:

- Print settings: - default -
- Filament: - default -
- Printer: - default FFF -
- Supports: None
- Infill: 20% Brim: ☐

Object manipulation

Name: gate_left.stl

	X	Y	Z	
Position:	100	100	7.24	mm
Rotate (relative):	0	0	0	°
Scale factors:	100	100	100	%
Size [World]:	8.74	94.46	14.48	mm

☐ Inches

5667.12 mm³ = 5.66712 cm³
ABS Filament Standard Average
Density = 1.05 g/cm³
Mass = Density*Volume = 1.05*5.66712
Each gate weighs 5.950 grams
Cost of printing one ABS part, as per
MAE3780 RPL Guide: \$1.50 + \$0.50/g
Total cost of gates (two) =
2*(\$1.50 + \$0.50/g*5.950g)
\$8.95

Sliced Info

Used Filament (m)	2.36
Used Filament (mm ³)	5667.12
Estimated printing time: - normal mode	24m

Export G-code

Weight + Cost of (2) Arms Shown in BOM (Appendix A)

Legend

27.35
(91)

0.35
(1)

42596

Print settings:
- default -
Filament:
- default -
Printer:
- default FFF -
Supports: None
Infill: 20%
Brim: ☐

12653.59 mm³ = 12.6536 cm³
ABS Filament Standard Average
Density = 1.05 g/cm³
Mass = Density*Volume = 1.05*12.6536
Each arm weighs 13.28628 grams
Cost of printing one ABS part, as per
MAE3780 RPL Guide: \$1.50 + \$0.50/g
Total cost of arms (two) = 2*(\$1.50
+ \$0.50/g*13.28628g)

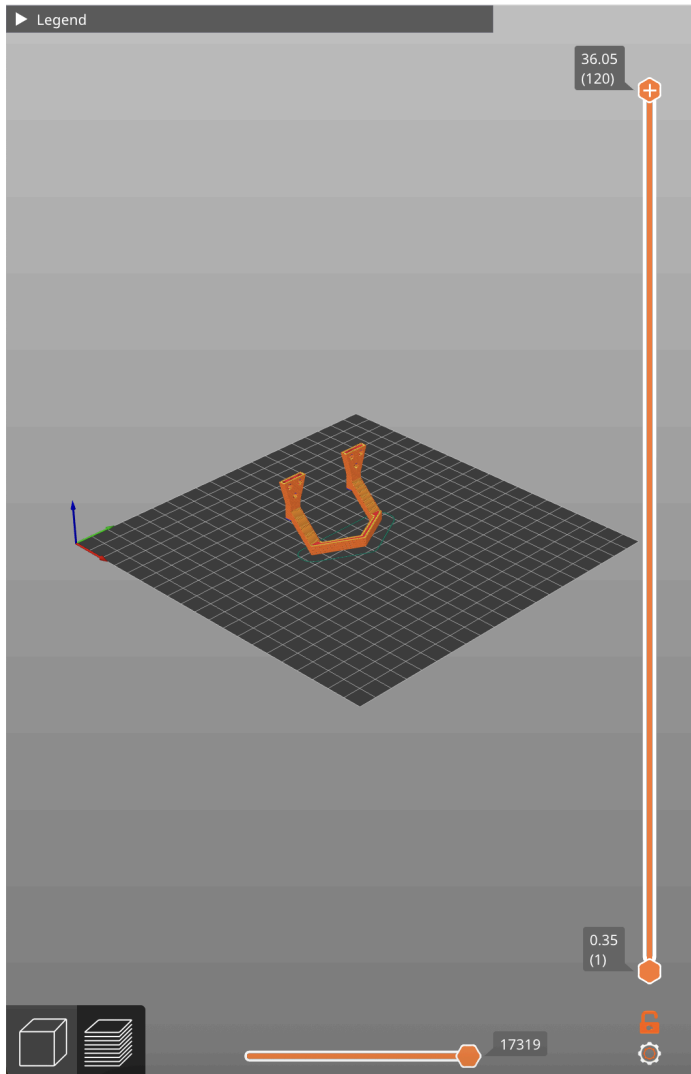
\$16.29

Sliced Info
Used Filament (m) 5.26
Used Filament (mm³) 12653.59
Estimated printing time:
- normal mode 1h5m

Export G-code

WhatsApp

Weight + Cost of Front Guard Shown in BOM (Appendix A)



Print settings:

- default -
- Filament: - default -
- Printer: - default FFF -
- Supports: None
- Infill: 20% Brim: ☐

Object manipulation

Name: **Front_guard.stl**

	X	Y	Z	
Position:	89.86	100	18	mm
Rotate (relative):	0	0	0	°
Scale factors:	100	100	100	%
Size [World]:	32	58.03	36	mm

☐ Inches

3432.03 mm³ = 3.432 cm³
PLA Filament Standard Average
Density = 1.24 g/cm³
Mass = Density*Volume = 1.24*3.432
The front guard weighs 4.256 grams
Cost of printing one PLA part, as per
MAE 3780 RPL Guide: \$1.00 + \$0.40/g
Total cost of front guard =
\$1.00 + \$0.40/g*4.256g

\$2.70

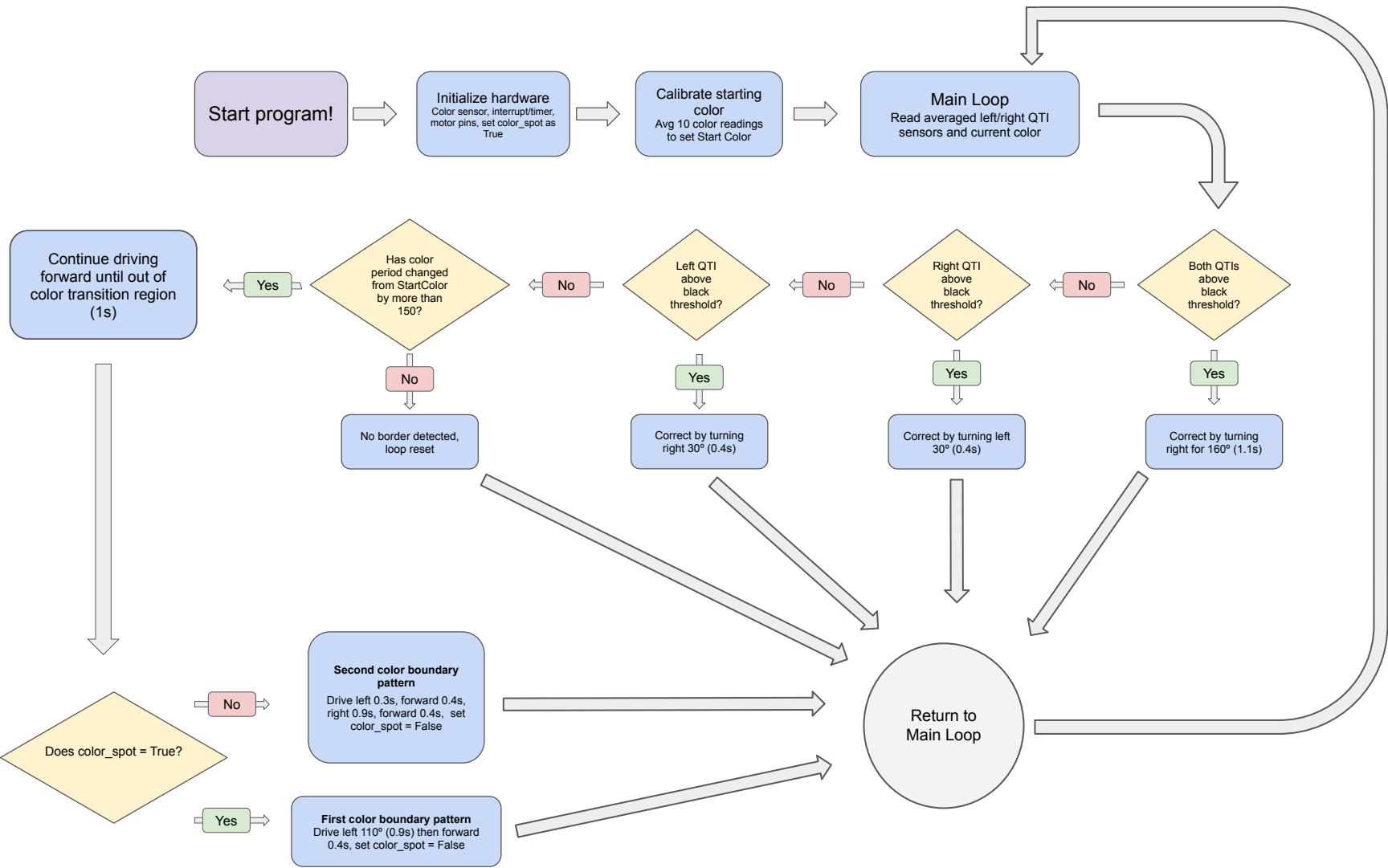
Sliced Info

Used Filament (m)	1.43
Used Filament (mm ³)	3432.03
Estimated printing time: - normal mode	30m

Export G-code

WhatsA

Appendix D



Appendix E:

```
#include <avr/io.h>
#include <avr/interrupt.h> // Required for timers and pin interrupts to run
// without pausing the main program
#include <stdlib.h> // Required for the abs() function

// Navigation States
// This was used initially, primarily for milestone 3. We kept for milestone 4 but as
// we continued refining our code and our approach, these don't really serve a purpose
// anymore
#define SEEK_DIVIDER 0
// #define TURN_180 1
// #define RETURN_HOME 2
#define FINISHED 3

// Initializing all functions and variables
void initColor();
int getColor();
uint16_t readQTI_right();
uint16_t readQTI_left();
volatile uint16_t startTime = 0;
volatile uint16_t capturedPeriod = 0;
bool Color_spot = true;
void drive_forward();
void turn_left();
void turn_right();
void stop();

//volatile uint16_t timerValue = 0;
//This is QTI readings
uint8_t confirmationCount = 0;

// Color Sensor Interrupt on Port B, Pin 8 (PB1)
// Corresponds to PCINT0
ISR(PCINT0_vect) {
    if (PINB & 0b00000001) {
        startTime = TCNT1; // Rising edge: Reset timer
    } else {
        capturedPeriod = TCNT1 - startTime; //Capture period when on falling edge
    }
}
```

```

int main(void) {
    init(); // Required for delay() to function
    initColor(); //Initialize color sensor
    // Motor Setup: Pins 10, 11, 12, 13 (PB3-PB7) as Output
    DDRD |= 0b00111100;

    uint8_t state = SEEK_DIVIDER;

    // Averaging initial reading to establish the "Home" color
    long sum = 0;
    int numSamples = 10;

    for (int i = 0; i < numSamples; i++) {
        sum += getColor();
        delay(20); // Small delay between samples for independence
    }

    int startColor = sum / numSamples; //divide the sum by the number of samples taken
    //for avg value

    // the main loop continues indefinitely as we never set state = finished.
    while (state != FINISHED) {

        uint16_t qti_left = getAverageQTI_left(); // gets averaged left qti reading
        uint16_t qti_right = getAverageQTI_right(); // gets averaged right qti reading
        int currentColor = getColor(); // gets current color

        if (state == SEEK_DIVIDER) {
            if (qti_left > 8000 && qti_right > 8000) { // Hits one of the black borders
                // square on
                turn_right();
                delay(1100); //didn't want full 180, 1100 is about 150° for
                // stochasticity
            }
            else if (qti_left > 8000){ // if only left qti hits border, turn right
                turn_right();
                delay(400);
                confirmationCount = 0;
            }
            else if (qti_right > 8000){ // if only right qti hits border, turn left
                turn_left();
                delay(400);
            }
        }
    }
}

```

```

    }

    // Check if the color sensor period changed significantly
    else{

        if (abs((currentColor) - startColor) > 150){
            drive_forward(); // didn't want the turn to be immediate, wanted it
// to go forward another foot or two for stochasticity purposes
            delay(1000);
            if (Color_spot) { // if first (or odd) time hitting color border,
// turn left (towards blocks) and drive forward.
                turn_left();
                delay(900);
                drive_forward();
                delay(400);
                startColor = getColor(); // reset start color, so the opposite
// color is now the home color
                Color_spot = true;
            }
            else if (!Color_spot) { // if second (or even) time hitting color
// border, make a left turn, drive forward, make a right turn drive forward... for
// stochasticity
                turn_left();
                delay(300);
                drive_forward();
                delay(400);
                turn_right();
                delay(900);
                drive_forward();
                delay(400);
                startColor = getColor(); // reset start color, so the opposite
// color is now the home color
                Color_spot = false;
            }
        }
    }
}
}
}

```

```

// --- Sensor Logic Functions ---

void initColor() {
    // Set Pin 8 (PB0) as Input
    DDRB &= 0b11111110;
    // Initialize Pin Change Interrupt Group 0 (Port B)
    PCICR |= 0b00000001;
    // Set mask for Pin 8 (PCINT0 is Bit 0 of PCMSK0)
    // keep off initially
    PCMSK0 |= 0b00000001;

    // Initialize the Timer 1 (16-bit)
    TCCR1A = 0;
    TCCR1B = 0b00000001; // No prescaler (16MHz)

    sei(); // Enable global interrupts
}

int getColor() {
    // Return period in microseconds (ticks * 0.0625 * 2)
    return (int)(capturedPeriod * 0.125);
}

uint16_t getAverageQTI_left() { //call the readQTI_left function a few times to
// average the qti reading and ensure it is actually hitting black (not just a weird
// change in lighting
    uint32_t sum = 0;
    uint8_t samples = 5; // Number of samples to average

    for (uint8_t i = 0; i < samples; i++) {
        sum += readQTI_left(); // calling the function to read the qti
    }
    return (uint16_t)(sum / samples);
}

uint16_t getAverageQTI_right() { // same as get average qti left
    uint32_t sum = 0;
    uint8_t samples = 5;

    for (uint8_t i = 0; i < samples; i++) {
        sum += readQTI_right();
    }
}

```

```

    }

    return (uint16_t)(sum / samples);
}

uint16_t readQTI_right() {
    uint16_t count = 0;
    // Charge Pin 6 (PD6) - Bit 6
    DDRD  |=  (1 << 6); // Set pin 6 as output
    PORTD |=  (1 << 6); // set pin 6 high
    delay(1);

    // Discharge Pin 6
    DDRD  &= ~(1 << 6); // Set pin 6 as input
    PORTD &= ~(1 << 6); // make sure pull-up is off

    // Time the decay of the capacitor
    while ((PIND & (1 << 6)) && (count < 10000)) {
        count++;
    }
    return count;
}

uint16_t readQTI_left() {
    uint16_t count = 0;
    // Charge Pin 7 (PD7) - Bit 7
    DDRD  |=  (1 << 7); // Set as output
    PORTD |=  (1 << 7); // Set pin 7 high
    delay(1);

    // Discharge Pin 7
    DDRD  &= ~(1 << 7); // Set as Input
    PORTD &= ~(1 << 7); // Ensure pull-up is off

    // time the delay of the capacitor
    while ((PIND & (1 << 7)) && (count < 10000)) {
        count++;
    }
    return count;
}

// --- Motor functions ---

```

```
void drive_forward(void) {  
    PORTD = (PORTD & 0b11000011) | 0b00010100;  
}  
  
void turn_right(void) {  
    // Right motor backward, left motor forward  
    PORTD = (PORTD & 0b11000011) | 0b00100100;  
}  
  
void turn_left(void) {  
    // Right motor forward, left motor backward  
    PORTD = (PORTD & 0b11000011) | 0b00011000;  
}
```